StudentEntryGUI Program

The `StudentEntryGUI.java` program collects a student's information through a window and saves it to a MySQL database.

Its overall flow is:

```
Open window → Enter information → Click Save Student
           → Validate input → Insert into MySQL → Show result
```

1. Import the required classes

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;
```

These packages provide:

Package	Used for
`javax.swing`	GUI components such as windows, buttons, and text fields
`java.awt`	Layout managers that arrange components
`java.sql`	JDBC connections and SQL statements

2. Define the window class

```
public class StudentEntryGUI extends JFrame {
```

`StudentEntryGUI` is your class. **`extends JFrame` means it inherits the features of a Swing window**, including a title bar, resizing, and a close button.

3. Store the database connection information

```
private static final String DB_URL =
        "jdbc:mysql://localhost:3306/university";
private static final String DB_USER = "your_username";
private static final String DB_PASSWORD = "your_password";
```

The URL identifies the database:

URL part	Meaning
`jdbc:mysql`	Connect using JDBC and the MySQL driver
`localhost`	MySQL runs on your computer
`3306`	MySQL's usual port
`university`	Database name

`final` prevents reassignment of these variables. `static` makes them belong to the class, and `private` limits direct access to this class.

4. Create the input fields and button

```
private final JTextField idField = new JTextField(20);
private final JTextField nameField = new JTextField(20);
private final JTextField majorField = new JTextField(20);
private final JButton saveButton = new JButton("Save Student");
```

The three text fields accept user input. The button starts the save operation.

The **`20` specifies a preferred width in text columns**, not a maximum of 20 characters. A layout manager can adjust the actual width.

These are instance fields so different methods in the class can access them.

5. Build the window in the constructor

```
public StudentEntryGUI() {
    super("Student Registration");
```

The constructor runs when the program creates a `StudentEntryGUI` object.

`super(...)` calls the `JFrame` constructor and sets the window title.

The constructor then creates the form:

```
JPanel form = new JPanel(new GridLayout(3, 2, 10, 10));

form.add(new JLabel("Student ID:"));
form.add(idField);
form.add(new JLabel("Name:"));
form.add(nameField);
form.add(new JLabel("Major:"));
form.add(majorField);
```

The grid contains three rows and two columns. Components fill it in the order they are added:

```
Student ID:   [ID field]
Name:         [Name field]
Major:        [Major field]
```

`GridLayout(3, 2, 10, 10)` specifies:

- **3 rows**
- **2 columns**
- **10 pixels** of horizontal space between columns
- **10 pixels** of vertical space between rows

Next, an outer panel holds the form and button:

```
JPanel panel = new JPanel(new BorderLayout(10, 10));

panel.setBorder(
        BorderFactory.createEmptyBorder(15, 15, 15, 15)
);

panel.add(form, BorderLayout.CENTER);
panel.add(saveButton, BorderLayout.SOUTH);
```

`BorderLayout` has five possible regions:

```
             NORTH
    WEST     CENTER     EAST
             SOUTH
```

The two numbers `(new BorderLayout(10, 10))` specify gaps between occupied regions:

- First **10**: horizontal gap of 10 pixels.
- Second **10**: vertical gap of 10 pixels.

The form occupies the center, and the button goes below it. The empty border supplies 15 pixels of padding on every side.

6. Connect the button to an action

```
saveButton.addActionListener(event -> saveStudent());
```

This registers an **event listener**.

When the user clicks the button, Swing calls:

```
saveStudent();
```

The expression `event -> saveStudent()` is a **lambda expression**: a short way to supply the action that should happen.

The constructor finishes configuring the window:

```
setContentPane(panel);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
pack();
setLocationRelativeTo(null);
```

These statements:

- Set the outer panel as the window's content.

- End the application when the window closes.
- Size the window to fit its components.
- Center the window on the screen.

7. Read and validate the input

Inside `saveStudent()`, the program converts the ID field's text into an integer:

```
final int studentId;

try {
    studentId = Integer.parseInt(idField.getText().trim());

    if (studentId <= 0) {
        throw new NumberFormatException();
    }
} catch (NumberFormatException exception) {
    showMessage("Enter a positive whole number for Student ID.");
    return;
}
```

For example:

```
"101" → 101
```

`trim()` removes leading and trailing spaces.

If the input is not a valid positive integer, the program shows a message. **`return` exits `saveStudent()`**, so no database operation occurs.

It then reads the name and major:

```
String name = nameField.getText().trim();
String major = majorField.getText().trim();
```

The following checks reject blank values or values longer than the database columns allow:

```
if (name.isEmpty() || major.isEmpty()) {
    showMessage("Please enter both a name and a major.");
    return;
}

if (name.length() > 100 || major.length() > 100) {
    showMessage("Name and major must each be 100 characters or fewer.");
    return;
}
```

8. Start the database work in the background

```
saveButton.setEnabled(false);
```

The program temporarily disables the button to prevent repeated clicks while saving.

It creates a background worker:

```
SwingWorker<String, Void> worker = new SwingWorker<>() {
    // doInBackground() and done() are defined here.
};
```

A database connection can take time. **`SwingWorker` lets the database operation run without freezing the window.**

The type arguments mean:

- `String`: the worker returns a string when successful.
- `Void`: this worker does not publish intermediate progress values.

The worker starts when this line executes:

```
worker.execute();
```

9. Connect to MySQL and insert the student

The worker's `doInBackground()` method contains the JDBC code:

```
String sql =
        "INSERT INTO students (student_id, name, major) "
        + "VALUES (?, ?, ?)";
```

The question marks are placeholders for values.

```
try (Connection connection = DriverManager.getConnection(
            DB_URL, DB_USER, DB_PASSWORD);
     PreparedStatement statement =
            connection.prepareStatement(sql)) {
```

`DriverManager.getConnection(...)` opens a database connection using the MySQL JDBC driver.

`prepareStatement(sql)` creates a parameterized statement. The program supplies the values separately:

```
statement.setInt(1, studentId);
statement.setString(2, name);
statement.setString(3, major);
```

Parameter positions start at 1.

Placeholder	Value

First ?	Student ID
Second ?	Name
Third ?	Major

Then:

```
statement.executeUpdate();
return "Student saved successfully!";
```

`executeUpdate()` executes the insertion. The connection uses JDBC's default **auto-commit**, so the insertion is committed automatically.

The `try (...)` structure is **try-with-resources**. It closes the statement and connection automatically, even if an exception occurs.

10. Display the result

After the background operation finishes, Swing calls `done()` on its GUI thread.

```
String message = get();
showMessage(message);
```

`get()` retrieves the result from `doInBackground()`. If the database operation failed, `get()` throws an `ExecutionException` instead.

On success, the program clears the fields:

```
idField.setText("");
nameField.setText("");
majorField.setText("");
idField.requestFocusInWindow();
```

The cursor returns to the ID field so the user can enter another student.

On failure, the program checks the underlying exception:

```
if (cause instanceof SQLException
        && ((SQLException) cause).getErrorCode() == 1062) {
    showMessage("That Student ID already exists.");
} else {
    showMessage("Unable to save: " + cause.getMessage());
}
```

MySQL error **1062** indicates a duplicate value in a unique key—for this table, a student ID that already exists.

Finally:

```
finally {
    saveButton.setEnabled(true);
}
```

The button becomes available again whether the operation succeeded or failed.

11. Show messages in a dialog

```
private void showMessage(String message) {
    JOptionPane.showMessageDialog(this, message);
}
```

This helper method displays a popup message attached to the current window. `this` refers to the current `StudentEntryGUI` object.

12. Start the application

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
            new StudentEntryGUI().setVisible(true));
}
```

`main()` is the program's entry point.

`SwingUtilities.invokeLater(...)` schedules window creation on Swing's **Event Dispatch Thread**, the thread responsible for GUI operations.

`new StudentEntryGUI()`

creates the window and runs its constructor.

`.setVisible(true)`

makes the window appear.

The program brings together three main ideas: **Swing builds the interface, event handling responds to the button, and JDBC saves the data in MySQL.**